

# Trabajo Práctico 2 — Catan

[7507/9502] Algoritmos y Programación III  
Curso 01 - Suárez  
Segundo cuatrimestre de 2025

## Integrantes

**Arroyo, Valentina Sol**

Padrón: 112294

vsarroyo@fi.uba.ar

**Arias, Rafaela**

Padrón: 112272

rparias@fi.uba.ar

**Marchesini, Carlota**

Padrón: 112099

cmarchesini@fi.uba.ar

**Baez, Brenda Carolina**

Padrón: 112496

bcbaez@fi.uba.ar

**López Angelini, Chiara**

Padrón: 111225

clopezan@fi.uba.ar

# Índice

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>1. Introducción</b>                           | <b>2</b>  |
| 1.1. Descripción del Juego . . . . .             | 2         |
| 1.2. Objetivos . . . . .                         | 2         |
| 1.3. Supuestos . . . . .                         | 2         |
| <b>2. Detalles de implementación</b>             | <b>3</b>  |
| 2.1. Herencia y Delegación . . . . .             | 3         |
| 2.1.1. Jerarquías de Herencia . . . . .          | 3         |
| 2.1.2. Delegación . . . . .                      | 3         |
| 2.2. Patrones . . . . .                          | 3         |
| 2.2.1. Patrón Factory . . . . .                  | 3         |
| 2.2.2. Patrón Strategy . . . . .                 | 4         |
| 2.2.3. Patrón Command . . . . .                  | 4         |
| 2.2.4. Patrón Visitor . . . . .                  | 5         |
| 2.2.5. Patrón Observer . . . . .                 | 5         |
| 2.2.6. Patrón Null Object . . . . .              | 6         |
| 2.2.7. Double Dispatch . . . . .                 | 6         |
| 2.2.8. Singleton . . . . .                       | 6         |
| 2.3. Arquitectura MVC . . . . .                  | 6         |
| 2.3.1. Modelo . . . . .                          | 7         |
| 2.3.2. Vista . . . . .                           | 7         |
| 2.3.3. Controlador . . . . .                     | 7         |
| 2.4. Principios SOLID . . . . .                  | 7         |
| 2.4.1. Single Responsibility Principle . . . . . | 7         |
| 2.4.2. Open/Closed Principle . . . . .           | 7         |
| 2.4.3. Liskov Substitution Principle . . . . .   | 8         |
| 2.4.4. Interface Segregation Principle . . . . . | 8         |
| 2.4.5. Dependency Inversion Principle . . . . .  | 8         |
| 2.5. Excepciones . . . . .                       | 8         |
| <b>3. Diagramas de Clases</b>                    | <b>9</b>  |
| <b>4. Diagramas de Secuencia</b>                 | <b>15</b> |
| <b>5. Diagramas de Paquetes</b>                  | <b>19</b> |

## 1. Introducción

El presente trabajo práctico consiste en el desarrollo de una aplicación completa del juego de estrategia Catan, implementada en Java y utilizando Programación Orientada a Objetos.

### 1.1. Descripción del Juego

**Catan** es un juego de mesa de estrategia y gestión de recursos donde los jugadores compiten por alcanzar 10 puntos de victoria mediante la construcción de poblados, ciudades y carreteras, así como la obtención de cartas especiales. El juego se desarrolla sobre un tablero compuesto por hexágonos de terreno que producen diferentes recursos (madera, ladrillo, lana, grano y mineral), los cuales los jugadores deben gestionar estratégicamente para expandir sus construcciones y superar a sus oponentes.

### 1.2. Objetivos

La aplicación debe incluir:

- Un modelo de clases que siga principios SOLID y patrones de diseño.
- Una interfaz gráfica intuitiva implementada con JavaFX.
- Pruebas unitarias e integrales mediante JUnit 5 y Mockito.

### 1.3. Supuestos

A continuación se detallan los supuestos realizados sobre el enunciado:

- **Número de jugadores:** La aplicación soporta 2, 3 y 4 jugadores. Se asume que el valor es seleccionado al inicio del juego y no puede modificarse durante la ejecución.
- **Ubicación de Puertos:** Los 9 puertos tienen una distribución fija sobre los bordes del tablero que se respeta en cada partida. La distribución está compuesta por 4 puertos genéricos (3:1) y 5 puertos especiales (2:1), uno por cada tipo de recurso: madera, ladrillo, lana, grano y mineral (el tipo de recurso asignado a cada puerto especial es aleatorio en cada partida). Una vez generada la distribución al inicio del juego, esta permanece fija durante toda la partida y no cambia.
- **Construcciones en Vértices de Borde de Mar:** Se permite construir poblados y ciudades en vértices ubicados en el borde del mar (que dan al mar), independientemente de que exista o no un puerto en esa ubicación.
- **Acceso a Puertos:** Un jugador puede comerciar con un puerto únicamente si posee una construcción (poblado o ciudad) ubicada en un vértice que tenga tal puerto o en un vértice adyacente a este.
- **Descarte de cartas:** Al salir el número 7 (ladrón), los jugadores que tengan más de 7 cartas, descartan la mitad (redondeando para abajo) de forma **aleatoria**.
- **Comercio Interno:** El Jugador decide el tipo y la cantidad de recursos a negociar con otro Jugador. La única limitación será que **efectivamente** cuenten con los recursos que intentan intercambiar.

## 2. Detalles de implementación

### 2.1. Herencia y Delegación

El proyecto utiliza herencia y delegación de manera estratégica para lograr un diseño flexible y mantenible:

#### 2.1.1. Jerarquías de Herencia

- **Terrenos:** La interfaz `Terreno` es implementada por clases concretas como `Bosque`, `Colina`, `Campo`, `Pastizal`, `Montaña` y `Desierto`. Cada una define su comportamiento específico de producción de recursos.
- **Recursos:** La interfaz `Recurso` es implementada por `Madera`, `Ladrillo`, `Lana`, `Grano` y `Mineral`. Se utiliza el patrón `Double Dispatch` para la comparación entre recursos mediante los métodos `coincideCon()`.
- **Construcciones:** La interfaz `Construccion` es implementada por `Poblado`, `Ciudad`, `Carretera` y `NullConstruccion` (patrón `Null Object`). Cada construcción define su costo, producción y puntos de victoria.
- **Cartas de Desarrollo:** La interfaz `CartaDesarrollo` (que extiende `Accion`) es implementada por `Caballero`, `PuntoVictoria`, `ProgresoConstruccion`, `ProgresoDescubrimiento` y `ProgresoMonopolio`.
- **Comercio:** Las interfaces `ComercioJugador` y `ComercioPuerto` son implementadas por clases específicas que encapsulan las diferentes modalidades de intercambio.

#### 2.1.2. Delegación

El proyecto hace uso extensivo de la delegación para distribuir responsabilidades:

- **Juego → Tablero:** La clase `Juego` delega al `Tablero` las operaciones relacionadas con producción de recursos, movimiento del ladrón y cálculo de rutas.
- **Jugador → Inventario:** El `Jugador` delega al `Inventario` toda la gestión de recursos (agregar, consumir, descartar).
- **Tablero → Hexágono:** El `Tablero` delega a cada `Hexagono` la producción de recursos según el número de dado lanzado.
- **Hexágono → Strategy:** Cada `Hexagono` delega su comportamiento de producción a su estrategia actual (`StrategySinLadron` o `StrategyConLadron`).

### 2.2. Patrones

#### 2.2.1. Patrón Factory

Se utilizó para la creación del tablero mediante la interfaz `TableroFactory` e implementación `TableroCatanFactory`.

**Aplicación:** La clase `TableroCatanFactory` encapsula toda la lógica compleja de inicialización del tablero, incluyendo:

- Generación aleatoria de 19 terrenos (4 bosques, 3 colinas, 4 pastizales, 4 campos, 3 montañas, 1 desierto).
- Generación aleatoria de 18 fichas numeradas (2-12 excluyendo 7).

- Creación de 54 vértices individuales.
- Asignación de vértices a hexágonos mediante **VerticeCatan**.
- Conexión de aristas entre vértices.
- Identificación y asignación aleatoria de los 9 puertos (4 genéricos y 5 especiales).

**Justificación:** El tablero es un objeto complejo que requiere múltiples pasos de inicialización y validación. Usar Factory permite encapsular toda esta complejidad, facilitando el testing (se puede mockear el tablero) y permitiendo diferentes implementaciones si en el futuro se quisieran tableros con configuraciones distintas (por ejemplo, tableros más pequeños o con variaciones).

### 2.2.2. Patrón Strategy

Se implementó para manejar el comportamiento variable de la producción de recursos en los hexágonos dependiendo de si el ladrón está presente o no.

**Aplicación:** Se definió la interfaz **StrategyHexagono** que declara el método **producirRecursos**.

- **Contexto:** La clase **Hexagono** mantiene una referencia a una estrategia actual (**estrategiaActual**).
- **Estrategias Concretas:**
  - **StrategySinLadron:** Implementa la lógica normal donde el hexágono distribuye recursos a sus vértices adyacentes.
  - **StrategyConLadron:** Implementa la lógica de bloqueo, donde el método de producción no realiza ninguna acción, simulando el efecto del ladrón.

**Justificación:** Este patrón permite alterar el comportamiento de un objeto **Hexagono** dinámicamente en tiempo de ejecución (al mover el ladrón) sin llenar el código de condicionales (*if-else*). Esto mejora la extensibilidad y mantiene el principio de responsabilidad única.

### 2.2.3. Patrón Command

Se implementó para encapsular las acciones del usuario como objetos, utilizando la interfaz **Action** y sus múltiples implementaciones concretas.

**Aplicación:** Cada jugada que modifica el estado del partido se modeló como un objeto propio:

- **Command:** La interfaz **Action**, que unifica la forma de ejecutar cualquier jugada.
- **Concrete Commands:** Clases como **AccionConstruir**, **AccionMoverLadron**, **AccionComprarCartaDesarrollar**, **AccionFinalizarTurno**, **AccionPrimeraColocacion**, **AccionSegundaColocacion** y **AccionRobarCarta**. Cada una guarda internamente los parámetros necesarios y sabe cómo validar y ejecutar esa lógica específica.
- **Receiver:** La clase **Juego**, que recibe la orden final a través del método **ejecutarAccion(Accion)**.
- **Invoker:** Los controladores de la vista (ej. **ConstruirControlador**), que crean el comando según el botón presionado y lo despachan al juego.
- **Extensión:** Las **CartaDesarrollo** también implementan **Action**, permitiendo ejecutarlas de la misma manera que cualquier otra acción del juego.

**Justificación:** En un juego como Catán con reglas complejas y múltiples fases por turno, centralizar toda la lógica en la clase **Juego** o en los controladores habría generado clases gigantes y difíciles de mantener (Anti-patrón *God Class*). Al usar Command, logramos que la lógica específica de cada jugada viva en su propia clase. Esto permitió que agregar nuevas funcionalidades, como las cartas de desarrollo, fuera tan simple como crear una nueva clase **Action** sin tener que modificar el flujo principal del juego ni los controladores de la interfaz gráfica.

### 2.2.4. Patrón Visitor

Se implementó mediante la interfaz **Visitante** y su implementación **VisitanteTerreno** para manejar la asignación de fichas numeradas a terrenos con comportamientos diferentes.

**Aplicación:** La lógica de asignación de fichas se encapsula en **VisitanteTerreno**, que es “aceptado” por cada terreno mediante el método **aceptar(Visitante)**. Esto permite:

- **Separación de responsabilidades:** La lógica de asignación de fichas no contamina las clases de terrenos (Bosque, Colina, Pastizal, Campo, Montaña, Desierto).
- **Comportamiento diferenciado:** El Desierto recibe un valor especial (0) sin necesidad de condicionales complejos, simplemente implementando **visitarDesierto()** diferente a **visitarTerrenoProductor()**.

**Justificación:** Evita la necesidad de usar **instanceof** o condicionales anidados para diferenciar entre Desierto y otros terrenos. El patrón Visitor invierte el control, permitiendo que cada terreno “acepte” al visitante y lo redirija al método apropiado. Esto facilita agregar nuevos comportamientos sin modificar las clases de terreno existentes.

### 2.2.5. Patrón Observer

Se implementó para desacoplar el modelo de la vista, permitiendo que la interfaz gráfica reaccione automáticamente a los cambios de estado en la lógica del juego sin que el modelo conozca detalles de la implementación visual.

**Aplicación:** Se definió la interfaz **Observador** con el método **actualizar()** y la clase abstracta **Observable** que gestiona la lista de suscriptores y la notificación.

- **Sujetos (Observables):** Las clases del modelo que sufren cambios de estado extienden de **Observable**.
  - **Jugador:** Notifica a los observadores cada vez que hay un cambio en su inventario (recibir/gastar recursos), cuando construye o cuando se modifican sus puntos de victoria.
  - **Dado:** Notifica a la vista cuando se lanza para que se actualice el valor mostrado.
  - **CambioTurnoControlador:** Notifica cuando cambia el turno para actualizar la interfaz.
- **Observadores:** Componentes de la vista que implementan la interfaz **Observador**.
  - **InfoJugador:** Observa a un objeto **Jugador** específico para actualizar en pantalla las etiquetas de recursos y puntos de victoria en tiempo real.
  - **VistaDados:** Observa a la clase **Dado** para mostrar las imágenes correspondientes a los valores obtenidos tras el lanzamiento.
  - **VistaTurnoActual:** Observa al **CambioTurnoControlador** para actualizar el indicador del jugador activo.

**Justificación:** El patrón Observer es fundamental para mantener la consistencia entre el estado del juego y lo que ve el usuario. Permite que múltiples componentes visuales dependan de un mismo objeto del modelo y se actualicen automáticamente, cumpliendo con el principio de separación de intereses (MVC) y evitando el acoplamiento fuerte entre la lógica de negocio y la interfaz de usuario.

### 2.2.6. Patrón Null Object

Se implementó para evitar verificaciones de nulos constantes y simplificar el código.

**Aplicación:**

- **NullConstruccion:** Implementa la interfaz **Construccion** con comportamientos neutros. Se utiliza como estado inicial de vértices y aristas vacíos, evitando verificaciones de null antes de invocar métodos.
- **NullPuerto:** Implementa la interfaz **ComercioPuerto** sin realizar ninguna acción. Se asigna a vértices que no tienen puerto, permitiendo intentar comerciar sin excepciones.

**Justificación:** Elimina la necesidad de verificaciones constantes de null (anti-patrón), simplifica el código y reduce la posibilidad de **NullPointerException**. Permite tratar uniformemente los casos donde hay o no hay una construcción/puerto presente.

### 2.2.7. Double Dispatch

Se utilizó en la comparación de recursos para determinar si dos recursos son del mismo tipo.

**Aplicación:** Cada clase de recurso implementa:

- Un método `coincideCon(Recurso otro)` que delega al recurso recibido
- Métodos específicos como `coincideConMadera()`, `coincideConLadrillo()`, etc.

**Ejemplo:**

```
// En Madera.java
public boolean coincideCon(Recurso otro) {
    return otro.coincideConMadera(this);
}

public boolean coincideConMadera(Madera madera) {
    return true;
}
```

**Justificación:** Evita el uso de `instanceof`, manteniendo el polimorfismo puro. Permite agregar nuevos tipos de recursos sin modificar las clases existentes, cumpliendo con el principio Open/-Closed.

### 2.2.8. Singleton

Se implementó para garantizar que exista una única instancia del gestor de música en toda la aplicación.

**Aplicación:**

- **Musica:** La clase mantiene una instancia única de **MediaPlayer** encargada de reproducir la música del juego. Cualquier parte de la interfaz puede solicitar reproducir, silenciar o cambiar la pista a través de la misma instancia global.

**Justificación:** Permite centralizar el control de la música, evitando la creación de múltiples reproductores que podrían superponer sonidos o consumir recursos innecesariamente. Asegura un comportamiento consistente a lo largo de todas las vistas del juego y simplifica la coordinación entre pantallas al utilizar siempre la misma fuente de audio.

## 2.3. Arquitectura MVC

El proyecto sigue una arquitectura Modelo-Vista-Controlador claramente definida:

### 2.3.1. Modelo

Contiene toda la lógica en el paquete `edu.fiuba.algo3.modelo`:

- **Juego y estado:** Juego, Jugador, Inventario, Dado
- **Tablero:** Tablero, Hexagono, Vertice, Arista
- **Elementos del juego:** Terreno, Recurso, Construccion, CartaDesarrollo
- **Comercio:** Interfaces y clases para gestionar intercambios
- **Excepciones:** Manejo de errores

### 2.3.2. Vista

Componentes visuales en el paquete `edu.fiuba.algo3.vistas`:

- Pantallas principales: VistaTituloJuego, VistaJuegoGeneral, VistaConstruir
- Componentes reutilizables: VistaTablero, InfoJugador, VistaDados
- Pop-ups: PopUpError, PopUpInformativo, PopUpAcercaDe

### 2.3.3. Controlador

Intermediarios en el paquete `edu.fiuba.algo3.controllers`:

- Controladores de flujo: JuegoControlador, CambioTurnoControlador
- Controladores de acciones: ConstruirControlador, ComercioControlador
- Controladores de fases: Paquete `fasesJuego` con controladores específicos para cada fase del juego

## 2.4. Principios SOLID

El diseño del proyecto respeta los principios SOLID:

### 2.4.1. Single Responsibility Principle

Cada clase tiene una única responsabilidad bien definida:

- **Inventario:** Gestiona exclusivamente los recursos de un jugador
- **Tablero:** Se encarga únicamente de la estructura y producción del tablero
- **Dado:** Solo maneja el lanzamiento de dados

### 2.4.2. Open/Closed Principle

El código está abierto a extensión pero cerrado a modificación:

- Agregar un nuevo tipo de terreno solo requiere implementar la interfaz `Terreno`
- Agregar una nueva carta de desarrollo solo requiere implementar `CartaDesarrollo`
- El patrón Strategy permite cambiar comportamientos sin modificar `Hexagono`



### 2.4.3. Liskov Substitution Principle

Las subclases pueden sustituir a sus clases base sin romper la funcionalidad:

- Cualquier **Construccion** puede ser tratada polimórficamente
- Cualquier **StrategyHexagono** puede intercambiarse sin afectar el comportamiento del hexágono

### 2.4.4. Interface Segregation Principle

Las interfaces son específicas y no obligan a implementar métodos innecesarios:

- **AccionControlador** vs **AccionesTableroControlador** vs **AccionHexagonoControlador**
- **ComercioJugador** vs **ComercioPuerto**

### 2.4.5. Dependency Inversion Principle

Las clases dependen de abstracciones, no de clases concretas:

- **Juego** depende de la interfaz **Accion**, no de implementaciones concretas
- **Hexagono** depende de **StrategyHexagono**, no de estrategias específicas
- Los controladores dependen de interfaces del modelo, no de implementaciones.

## 2.5. Excepciones

El programa cuenta con diversas excepciones que, no sólo detienen el proceso que lanza el error, sino que se utilizan para brindar información útil al usuario mediante la interfaz gráfica. Una vez capturado el error, los controladores lo manejan emitiendo un sonido y/o aviso escrito, explicando porqué esa acción no está permitida.

- **ComercioInvalidoError**: Se lanza dentro de **PuertoGenerico** y **PuertoEspecial** cuando se intenta comerciar con tasas diferentes a las aceptadas por el puerto.
- **ConstruccionInvalidaError**: Se lanza dentro de **Carretera**, **Poblado** y **Ciudad** cuando se intenta mejorar una construcción que no es propiedad del jugador de turno. También se captura esta excepción cuando se intenta construir una **Carretera** sobre un vértice, colocar una **Ciudad** donde ya existe una o construir una **Ciudad** sin tener un **Poblado** previo en esa ubicación.
- **NoHayCartasDesarrolloError**: Se lanza dentro de la clase **Juego** cuando se intenta comprar una carta de desarrollo pero ya no quedan en el mazo.
- **RecursosInsuficientesError**: Se lanza dentro de la clase **Inventario** cuando se intentan eliminar recursos que el jugador no posee. Esta eliminación puede ser por comercio, construcción o adquisición de **Cartas Desarrollo**.
- **ReglaDeDistanciaError**: Se lanza en **Vértice** cuando se intenta construir en una distancia menor a la permitida por las reglas del juego.
- **YaHayCarreteraError**: Se lanza dentro de la clase **Carretera** cuando se intenta construir en una arista ocupada.
- **YaHayCiudadError**: Se lanza dentro de la clase **Ciudad** cuando se intenta construir en un vértice ocupado.
- **YaHayPobladoError**: Se lanza dentro de **Poblado** cuando se intenta construir en un vértice ocupado.

### 3. Diagramas de Clases

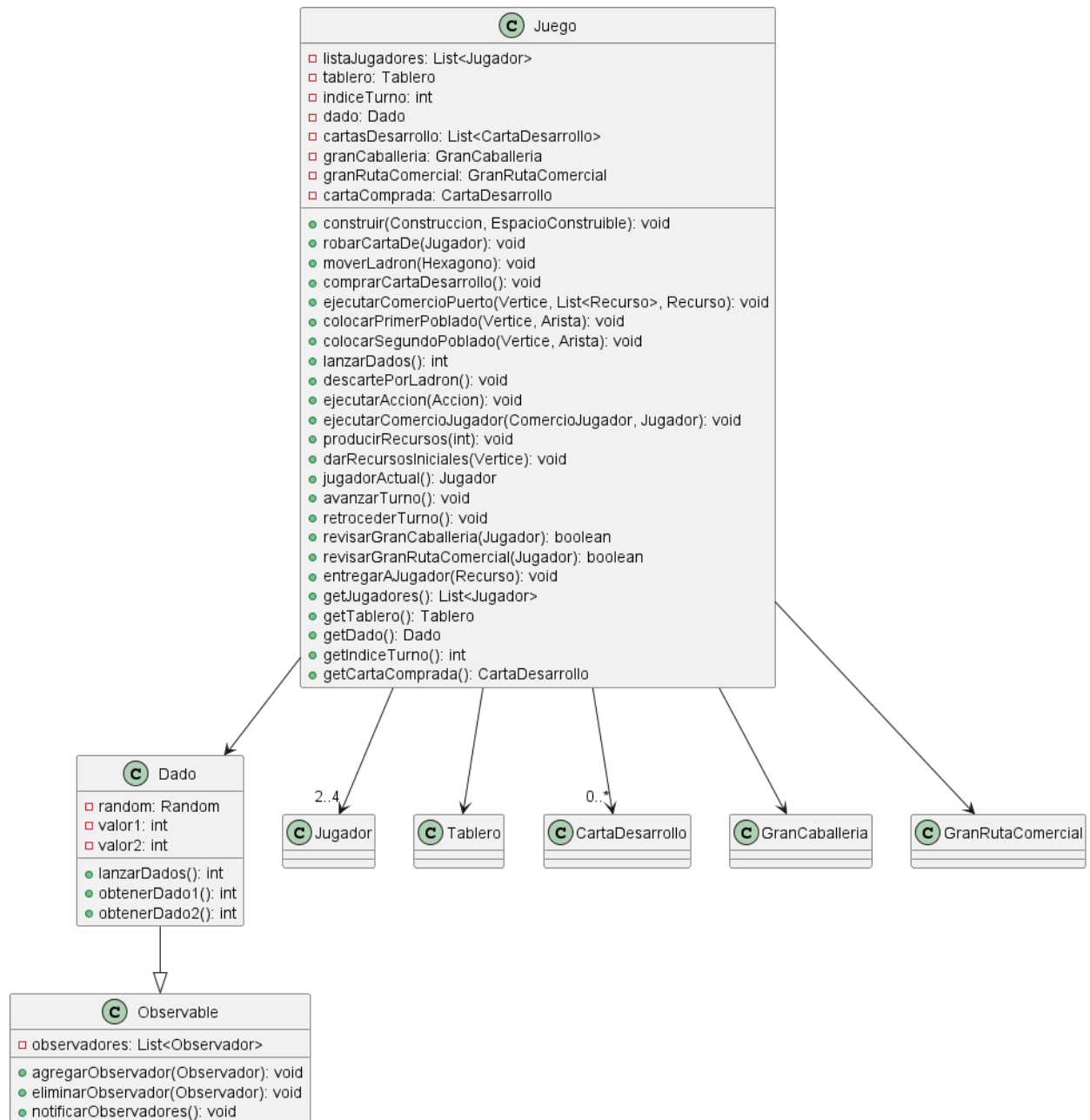


Figura 1: Diagrama de Juego.

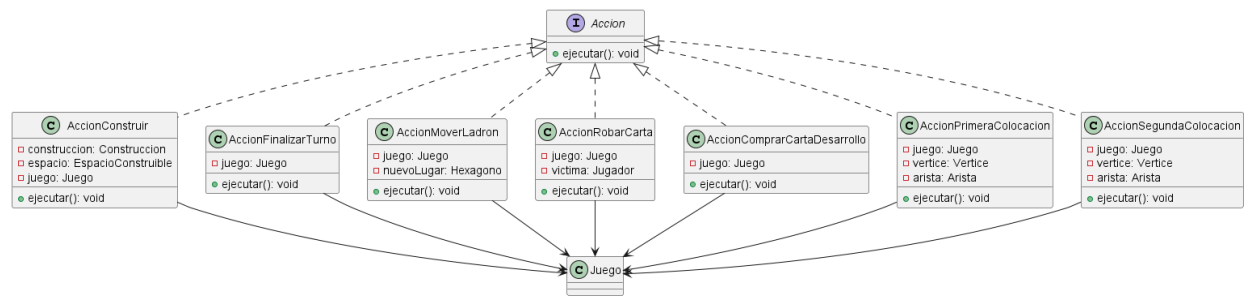


Figura 2: Diagrama de Juego Command.



Figura 3: Diagrama de Jugador.

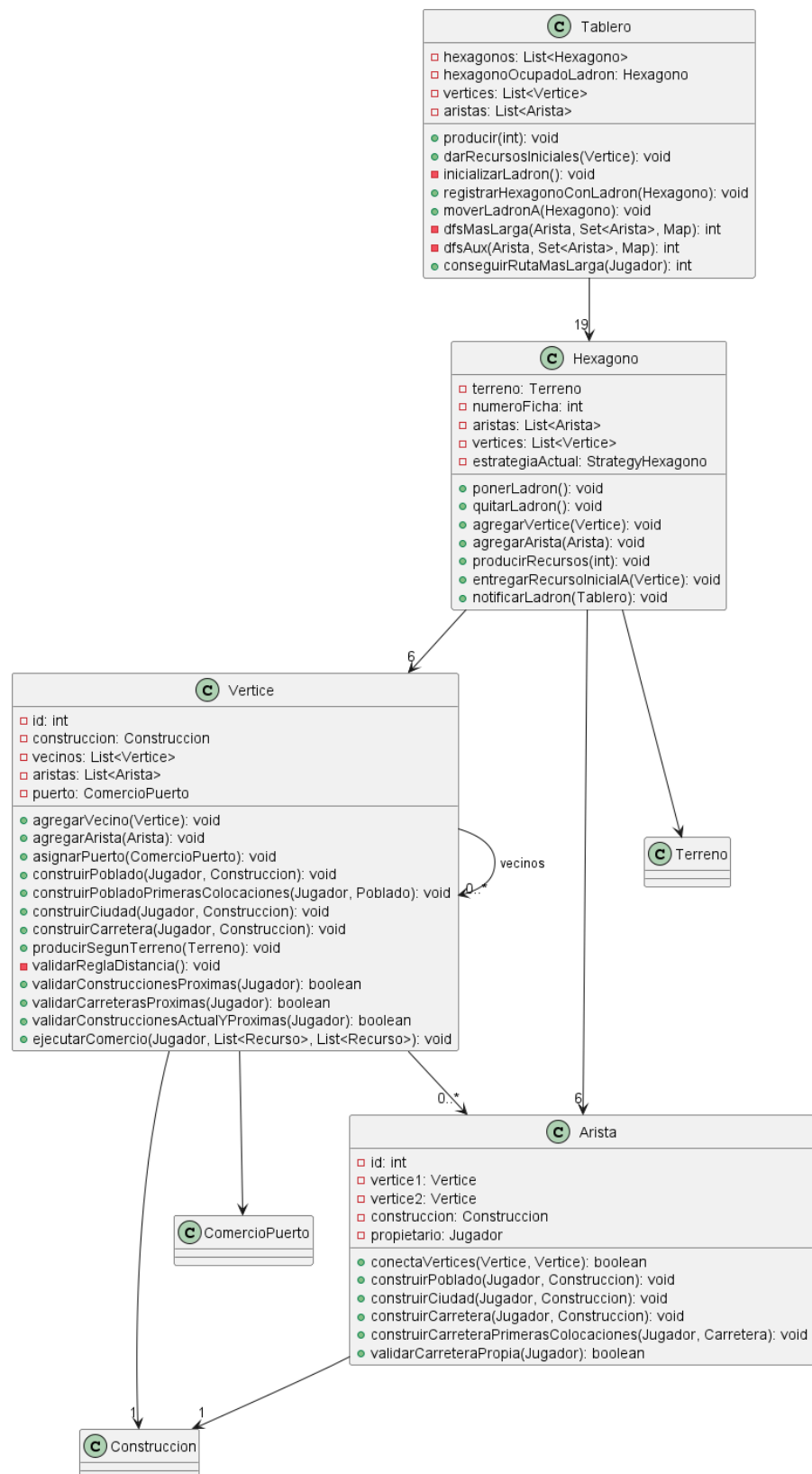


Figura 4: Diagrama de Tablero.

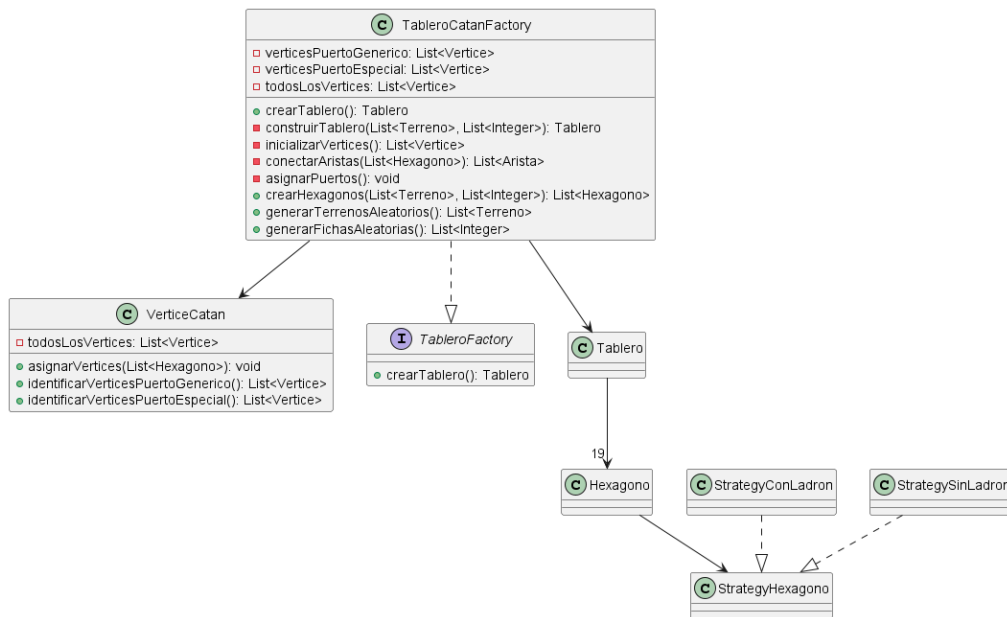


Figura 5: Diagrama de Tablero Factory.

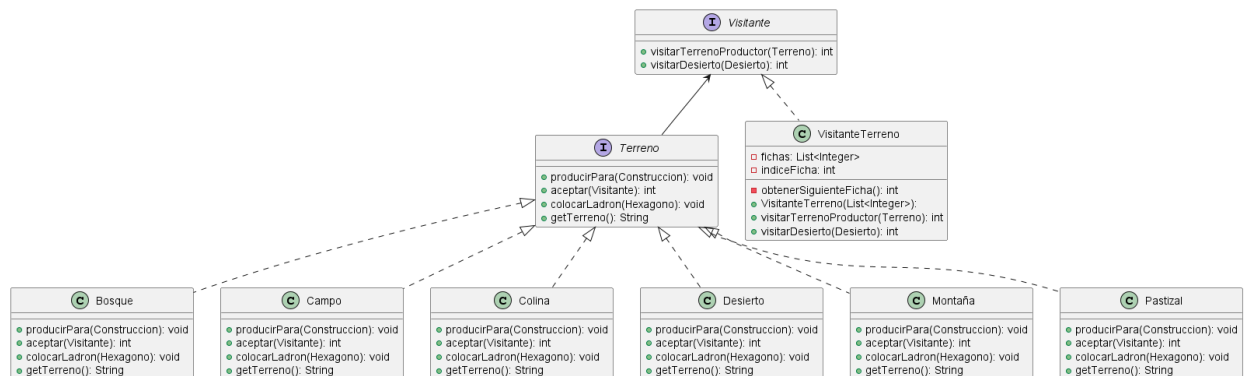


Figura 6: Diagrama de Terreno.

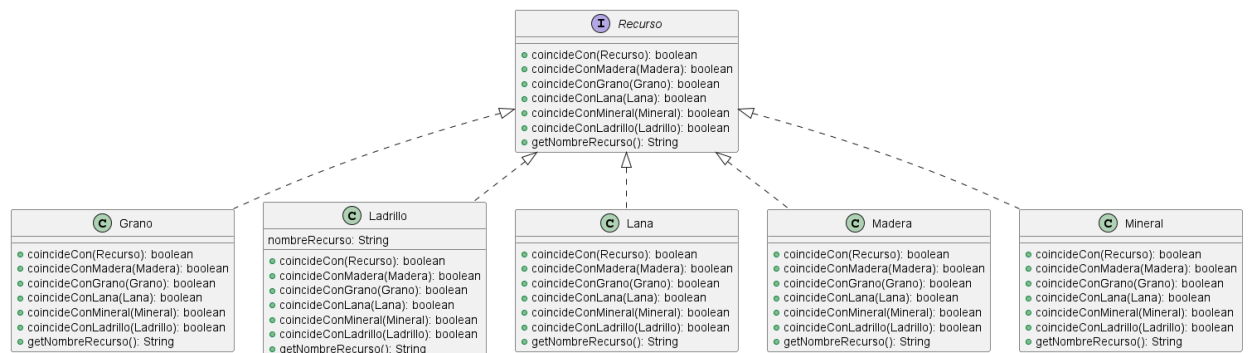


Figura 7: Diagrama de Recursos.

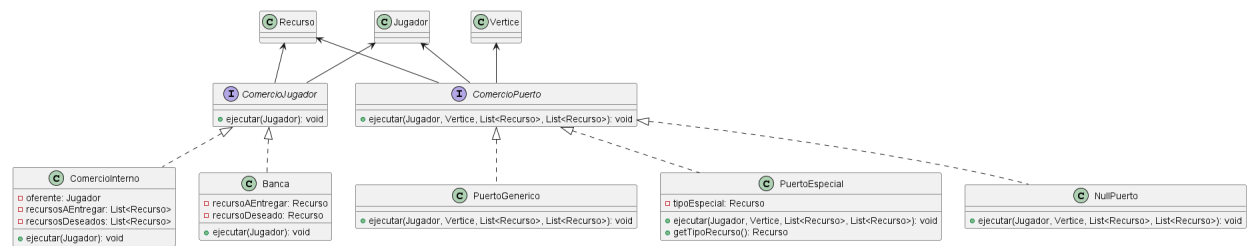


Figura 8: Diagrama de Comercio.

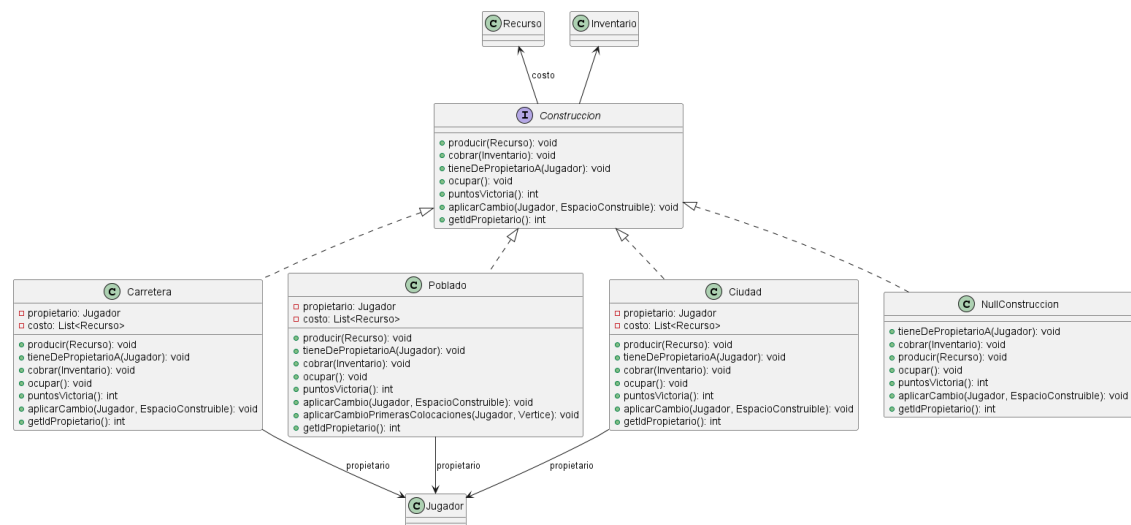


Figura 9: Diagrama de Construcciones.

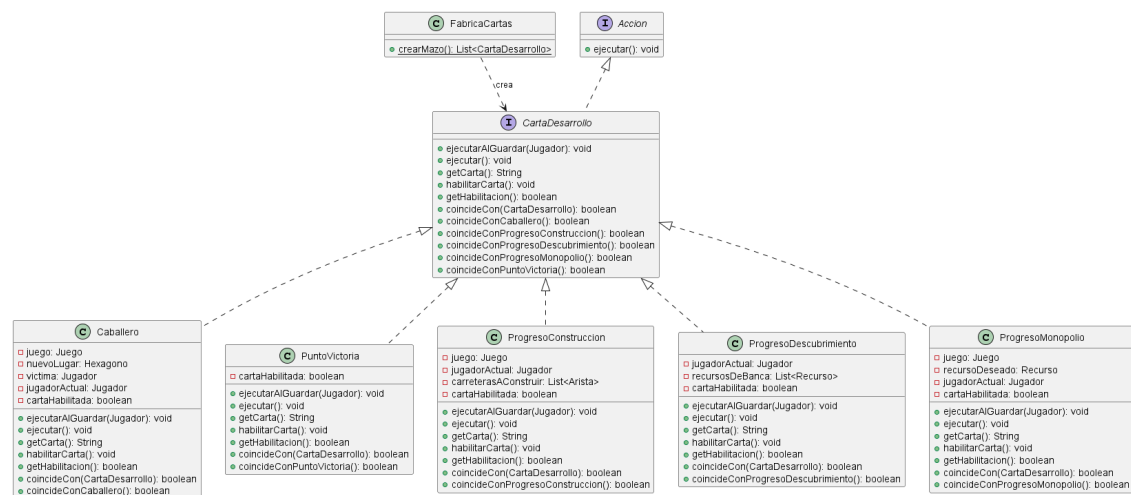


Figura 10: Diagrama de CartaDesarrollo.

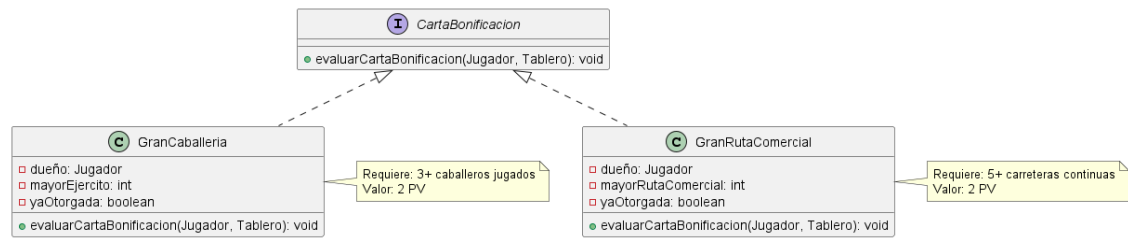


Figura 11: Diagrama de CartaBonificacion.

## 4. Diagramas de Secuencia

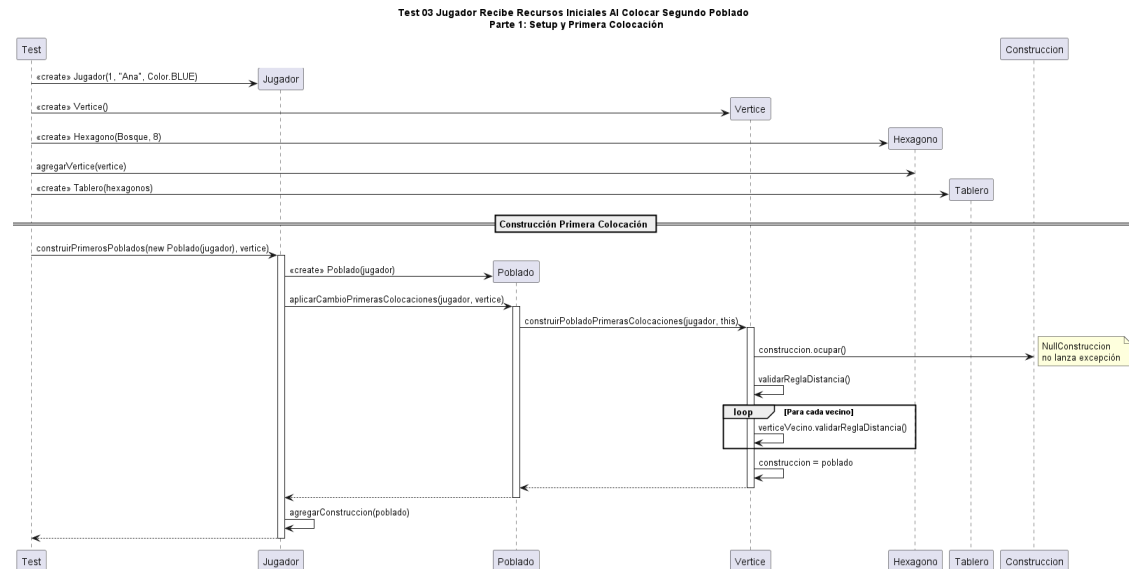


Figura 12: Diagrama de Secuencias del Test 03

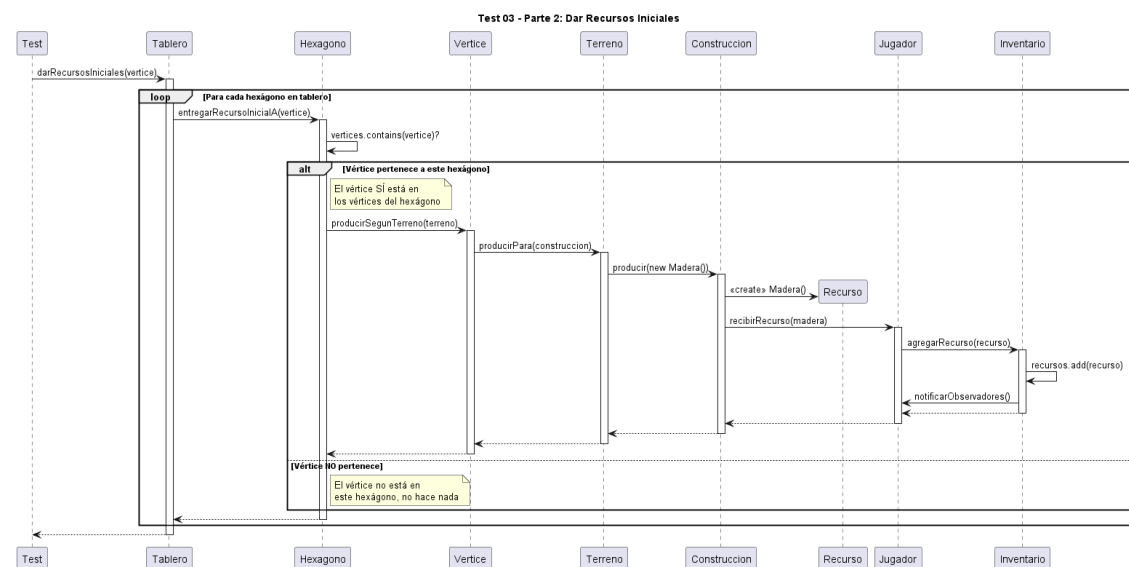


Figura 13



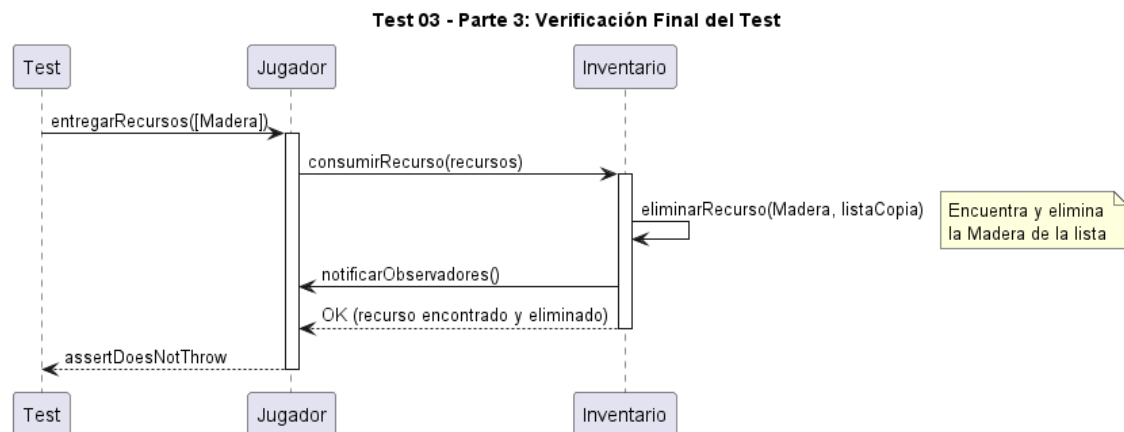


Figura 14

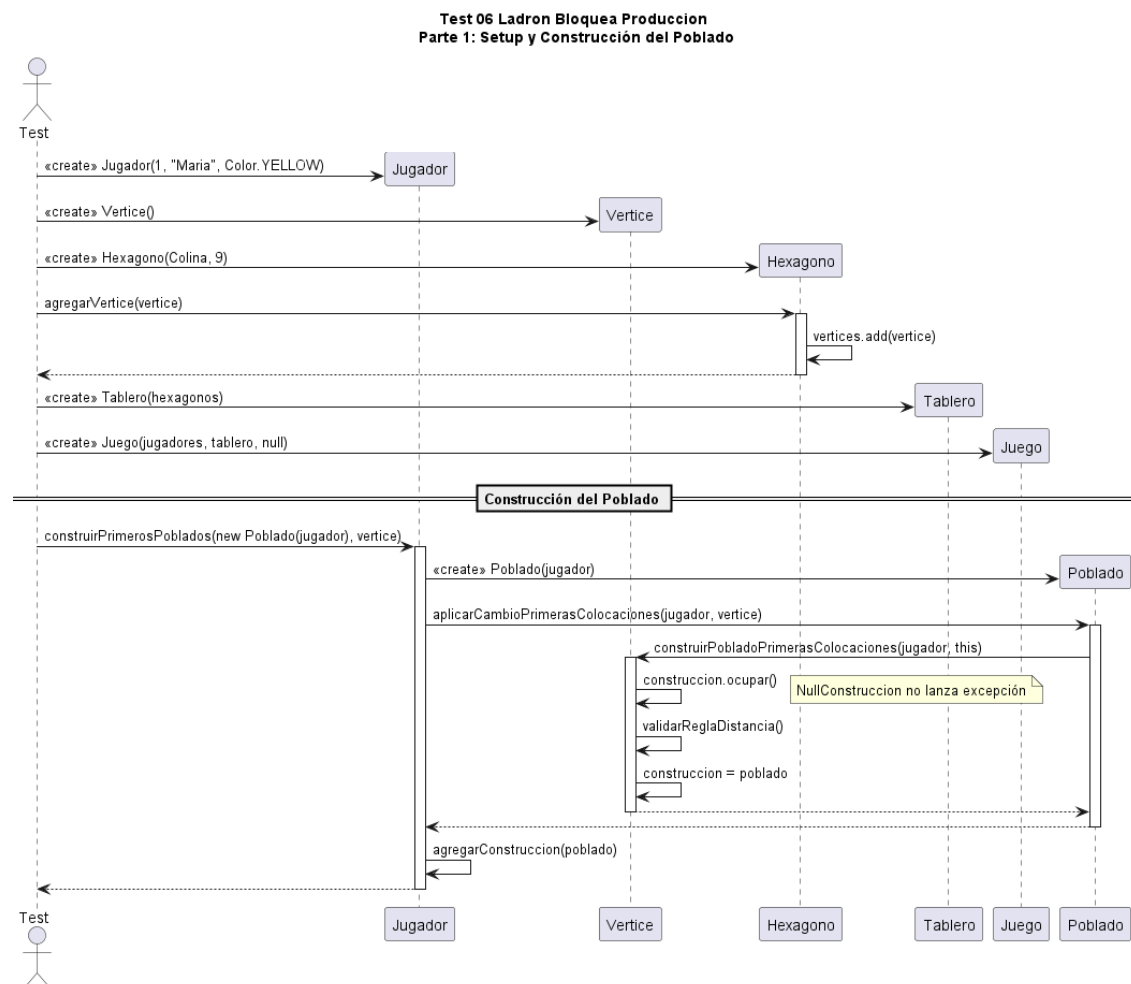


Figura 15: Diagrama de Secuencias del Test 06

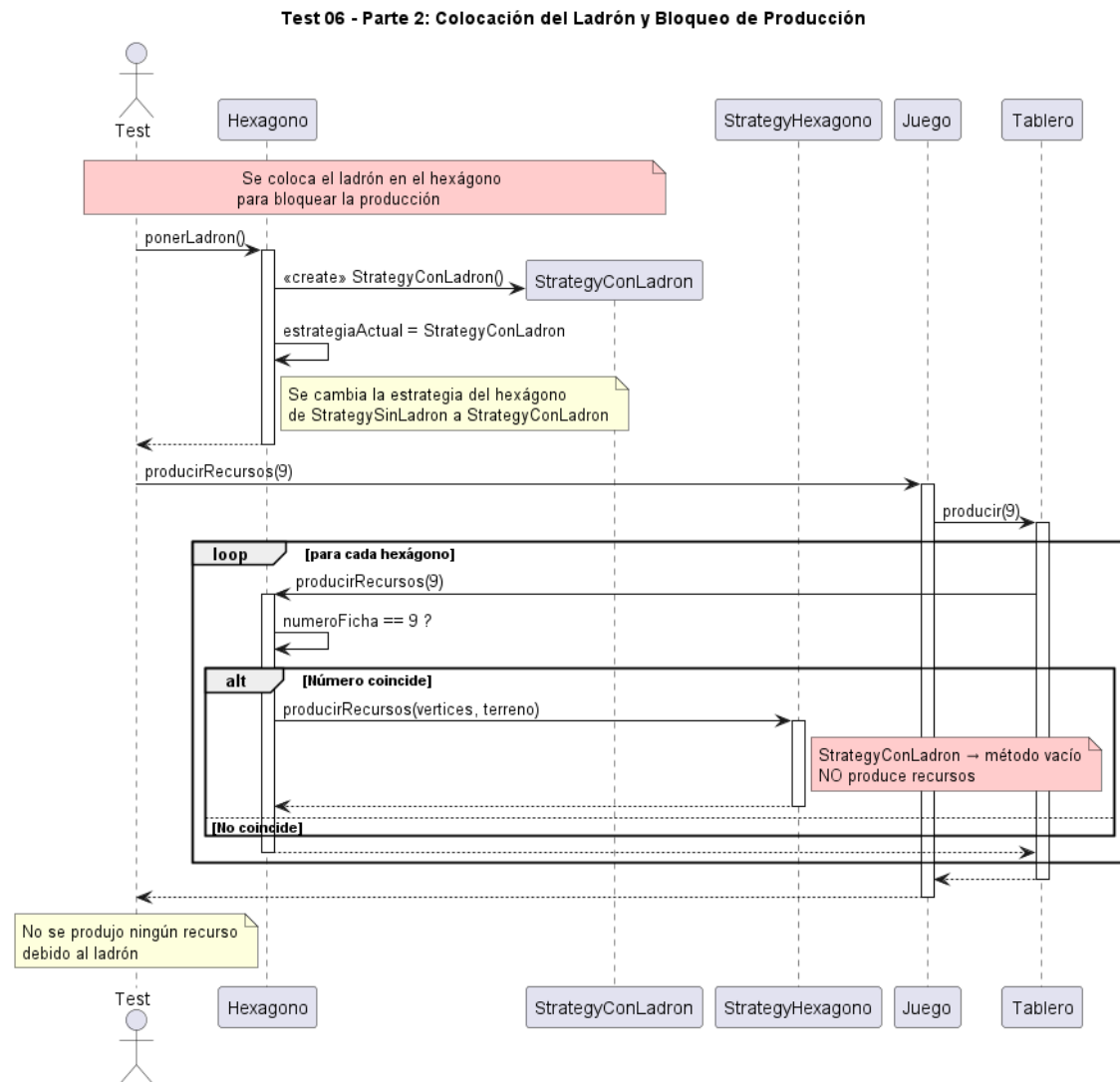


Figura 16

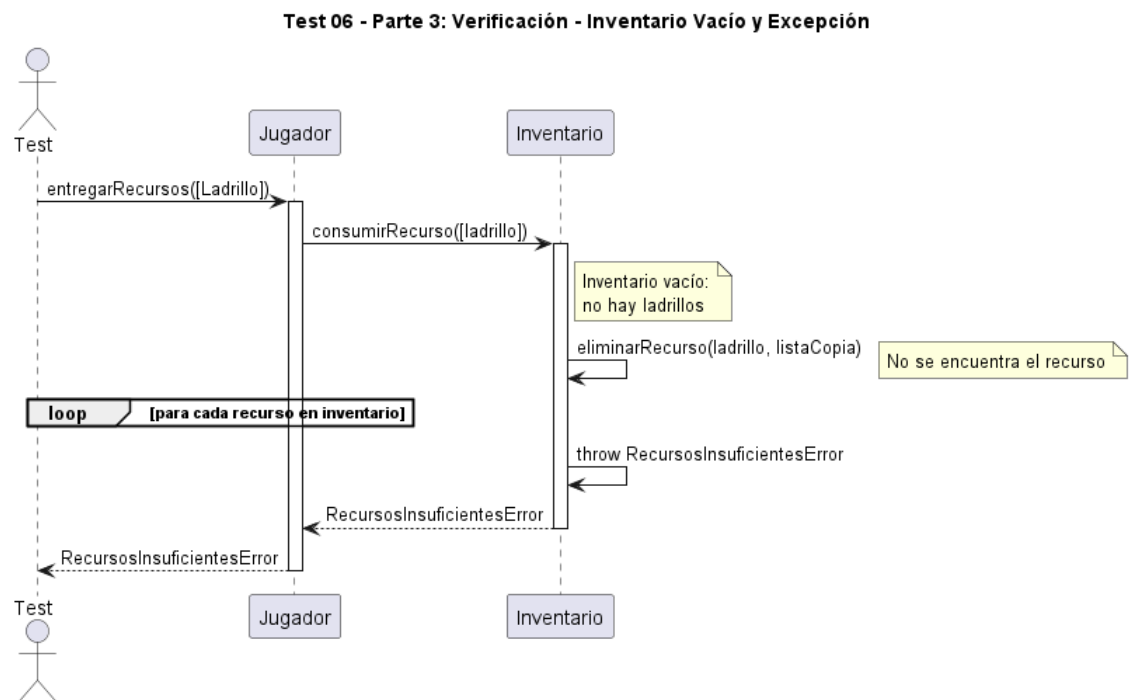


Figura 17

## 5. Diagramas de Paquetes

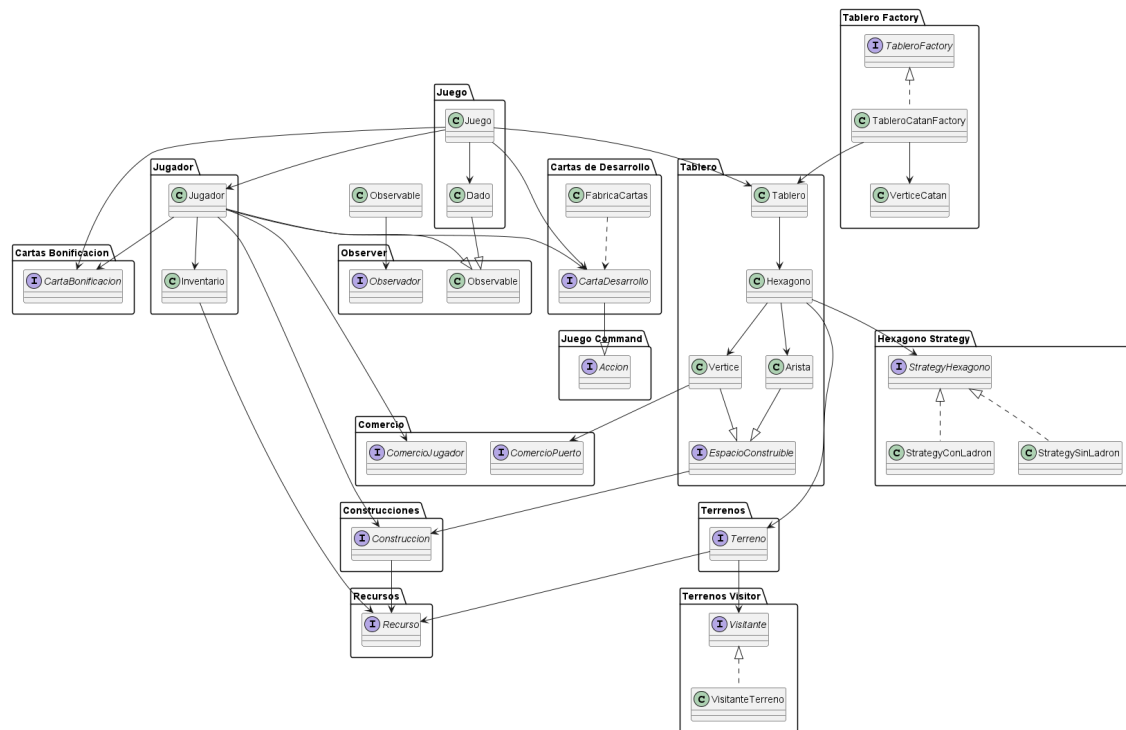


Figura 18: Diagrama de Paquetes